

What You Need:

1. **SonarQube Server** – You have already downloaded it. Make sure it is running.
2. **SonarScanner** – A command-line tool that sends analysis reports to SonarQube.
3. **Java & Maven Installed** – Since you have a `pom.xml` file, you should have Maven.

Start SonarQube Server

- Open a terminal and navigate to the SonarQube directory.

Start the server:

```
./bin/linux-x86-64/sonar.sh start # Linux/macOS
```

- Open `http://localhost:9000` in a browser and log in (default username/password: `admin/admin`)

METHOD 1(using sonar scanner) :

Step 1: Download and Install SonarScanner

SonarScanner is needed to analyze your Java project.

Install SonarScanner on RedHat

Wget

```
https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-7.0.2.4839-linux.zip
```

1. Extract the downloaded file:

```
# unzip sonar-scanner-7.0.2.4839-linux.zip
```

2. Move it to `/opt` for system-wide access:

```
# sudo mv sonar-scanner-7.0.2.4839-linux /opt/sonar-scanner
```

3. Add SonarScanner to your `PATH`:

```
# echo 'export PATH=$PATH:/opt/sonar-scanner/bin' >> ~/.bashrc
```

```
# source ~/.bashrc
```

4. Verify installation:

```
# sonar-scanner --version
```

Step 2: Configure SonarScanner for Your Project

Navigate to your Java project folder and create a **sonar-project.properties** file:

```
cd /path/to/your/java/project (where pom file exist or java project root directroy)
```

```
vi sonar-project.properties
```

```
sonar.projectKey=my-java-project

sonar.projectName=My Java Project

sonar.projectVersion=1.0

sonar.sources=src

sonar.language=java

sonar.java.binaries=target/classes

sonar.sourceEncoding=UTF-8

sonar.host.url=http://localhost:9000

sonar.login=your_sonar_token
```

Step 4: Run Code Analysis

Now, execute the SonarScanner command to analyze your Java code:

```
# mvn compile
```

```
# sonar-scanner
```

Step 5: View Results

1. Open **http://localhost:9000** in your browser.
2. Click on your **project name** to view:
 - Code quality reports
 - Bugs & vulnerabilities
 - Code smells & duplications

METHOD 2 (using maven-> mvn sonar:sonar) :

Step 1: Add SonarQube Plugin to `pom.xml`

Open your `pom.xml` file in the project root directory:

```
<build>

  <plugins>

<!-- SonarQube Scanner Plugin -->

    <plugin>

      <groupId>org.sonarsource.scanner.maven</groupId>

      <artifactId>sonar-maven-plugin</artifactId>

      <version>3.9.1.2184</version>

    </plugin>

  </plugins>

</build>

<reporting>

  <plugins>

<!-- SonarQube Reports -->

    <plugin>

      <groupId>org.sonarsource.scanner.maven</groupId>

      <artifactId>sonar-maven-plugin</artifactId>

      <version>3.9.1.2184</version>

    </plugin>

  </plugins>

</reporting>
```

Step 2: Also add this to `pom.xml`

```
<properties>

  <maven.compiler.source>17</maven.compiler.source>

  <maven.compiler.target>17</maven.compiler.target>

  <sonar.host.url>http://192.168.56.15:9000</sonar.host.url>

  <sonar.login>your_sonar_token</sonar.login>

</properties>
```

Step 3: Run the Maven Command for SonarQube Analysis

```
# mvn clean sonar:sonar
```